Neurônio Artificial I

Uma Exploração Estrutural da Unidade Fundamental das Redes Neurais

Autor: Marcos júnior Lemes Ferreira
Ano: 2026

Resumo

Este artigo oferece uma exploração conceitual, matemática e prática do neurônio artificial, a unidade básica das redes neurais artificiais. Inspirado no modelo biológico proposto por McCulloch e Pitts em 1943, o neurônio artificial serve como bloco de construção para sistemas de inteligência artificial complexos, como modelos de linguagem generativos. O foco é fornecer uma compreensão clara de sua estrutura interna, incluindo a combinação linear de entradas, a aplicação de funções de ativação e sua interpretação geométrica. Além disso, são incluídos exemplos numéricos, implementações em Python e discussões sobre como esses neurônios se encadeiam em camadas para formar redes profundas. Com base em referências clássicas da literatura, o texto enfatiza como operações simples — multiplicação, soma e não-linearidades — permitem a emergência de comportamentos complexos em larga escala, sem adentrar em algoritmos de treinamento.

Sumário

1 Introdução **1**

2 História Breve do Neurônio Artificial **1**

3 Como Modelos Representam Dados **1**

4 Formulação Matemática do Neurônio **2**

4.1 Combinação Linear 2

4.2 Função de Ativação 2

5 Exemplos de Funções de Ativação Comuns **2**

6 Exemplo Numérico **3**

7 Interpretação Geométrica **4**

8 Implementação em Python **4**

9 Camadas e Encadeamento **6**

10 Grandes Modelos **6**

11 Limitações e Considerações Práticas **7**

12 Conclusão **7**

1 Introdução

Modelos modernos de linguagem, como o GPT-4 da OpenAI ou o Claude da Anthropic, são compostos por bilhões de neurônios artificiais organizados em camadas profundas. Esses sistemas demonstram capacidades impressionantes, como gerar texto coerente, responder a perguntas complexas e resolver problemas abstratos em domínios variados. No entanto, essa sofisticação aparente pode mascarar a simplicidade subjacente: toda a complexidade emerge de uma unidade fundamental extremamente básica, o neurônio artificial.

Inspirado vagamente no neurônio biológico, o neurônio artificial processa entradas numéricas para produzir saídas, permitindo que redes neurais modelem relações não-lineares em dados. Como destacado por Goodfellow et al. (2016), essa unidade é o cerne da aprendizagem profunda, onde a composição de operações elementares cria representações hierárquicas de informações. Este artigo explora sua estrutura de forma acessível, visando desmistificar como sistemas inteligentes surgem de componentes simples.

2 História Breve do Neurônio Artificial

O conceito de neurônio artificial remonta a 1943, quando Warren McCulloch e Walter Pitts propuseram um modelo lógico simplificado do neurônio biológico, capaz de realizar operações booleanas. Esse trabalho seminal, publicado em "A Logical Calculus of the Ideas Immanent in Nervous Activity", estabeleceu as bases para a computação neural, mostrando que redes de neurônios threshold (limiar) poderiam simular qualquer função lógica finita.

Em 1958, Frank Rosenblatt introduziu o Perceptron, uma implementação prática que adicionava pesos ajustáveis e bias, permitindo aprendizado supervisionado via regra de atualização. Embora limitado a problemas linearmente separáveis, o Perceptron pavimentou o caminho para redes multicamadas. Trabalhos posteriores, como os de Rumelhart et al. (1986) sobre backpropagation, superaram essas limitações, mas a essência estrutural permaneceu inalterada. Essa evolução histórica reforça a robustez do modelo: simples em design, mas poderoso em composição (Bishop, 2006).

3 Como Modelos Representam Dados

Antes de mergulhar no neurônio, é essencial entender como os modelos processam informações. Diferentemente dos humanos, que interpretam texto ou imagens de forma qualitativa, os modelos computacionais convertem tudo em representações numéricas. Isso garante consistência e permite operações matemáticas eficientes.

A unidade básica é o **vetor**, uma lista ordenada de números que codifica características da entrada. Por exemplo, um vetor pode representar pixels de uma imagem ou embeddings de palavras:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Cada x_i captura uma dimensão específica, como intensidade de cor ou frequência de uma palavra.

Ao lidar com múltiplas amostras, usamos **matrizes**. Uma matriz X agrupa vetores em colunas ou linhas:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}$$

Em modelos avançados, como transformers, as estruturas evoluem para **tensores** multidimensionais, que capturam relações espaciais ou temporais (por exemplo, em vídeos ou sequências de texto). Tudo o que o modelo "aprende" é armazenado como ajustes nesses números, otimizando representações em espaços vetoriais de alta dimensionalidade. Essa abordagem vetorial é crucial, pois permite que neurônios operem uniformemente sobre qualquer tipo de dado, desde que convertido numericamente (Haykin, 2009).

4 Formulação Matemática do Neurônio

O neurônio artificial executa duas etapas principais: uma combinação linear das entradas seguida de uma transformação não-linear.

4.1 Combinação Linear

A entrada é ponderada por pesos e somada a um bias:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Onde: - $\mathbf{x} = [x_1, x_2, \ldots, x_n]$: Vetor de entradas. - $\mathbf{w} = [w_1, w_2, \ldots, w_n]$: Vetor de pesos, que determinam a importância de cada entrada. - b: Bias, um termo constante que ajusta o limiar de ativação.

Essa operação é equivalente a um produto interno: $z = \mathbf{w}^T\mathbf{x} + b$. Os pesos e bias são parâmetros aprendidos durante o treinamento, permitindo que o neurônio se adapte aos dados.

4.2 Função de Ativação

Para introduzir não-linearidade — essencial para modelar relações complexas — aplica-se uma função f:

$$a = f(z)$$

Sem não-linearidade, redes multicamadas colapsariam em uma única transformação linear, limitando sua expressividade (Goodfellow et al., 2016). A ativação permite que o neurônio "decida" se ativar ou não, simulando o disparo neuronal biológico.

5 Exemplos de Funções de Ativação Comuns

Além da função degrau simples usada em exemplos básicos, várias funções são empregadas em redes modernas para melhorar o treinamento e o desempenho:

- **Função Degrau (Step)**:

$$f(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{caso contrário} \end{cases}$$

Útil para classificações binárias, mas não diferenciável, limitando o uso em otimização baseada em gradientes.

- **Sigmoide**:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Produz saídas entre 0 e 1, ideal para probabilidades. No entanto, sofre de vanishing gradients em valores extremos.

- **Tangente Hiperbólica (Tanh)**:

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Similar à sigmoide, mas com saídas entre -1 e 1, centrando os dados para camadas subsequentes.

- **ReLU (Rectified Linear Unit)**:

$$f(z) = \max(0, z)$$

Eficiente computacionalmente e mitiga vanishing gradients. Variantes como Leaky ReLU ($f(z) = \max(0.01z, z)$) evitam "neurônios mortos".

Essas funções são escolhidas com base no problema: ReLU é padrão em redes convolucionais para imagens, enquanto softmax é usada em saídas para classificações multiclasse (Bishop, 2006).

Aqui está um diagrama típico de funções de ativação:

6 Exemplo Numérico

Considere um neurônio com entradas, pesos e bias:

$$\mathbf{x} = [1, 0, 1], \quad \mathbf{w} = [0.5, -0.3, 0.8], \quad b = 0.2$$

Cálculo da combinação linear:

$$z = (1 \cdot 0.5) + (0 \cdot -0.3) + (1 \cdot 0.8) + 0.2 = 0.5 + 0 + 0.8 + 0.2 = 1.5$$

Aplicando a função degrau:

$$a = f(1.5) = 1 \quad (\text{pois } 1.5 > 0)$$

Agora, usando ReLU:

$$a = \max(0, 1.5) = 1.5$$

E sigmoide:

$$a = \frac{1}{1 + e^{-1.5}} \approx 0.817$$

Esses exemplos ilustram como diferentes ativações alteram a saída, afetando a interpretabilidade e o fluxo de gradientes em redes maiores.

7 Interpretação Geométrica

Geometricamente, a equação $\sum_{i=1}^{n} w_i x_i + b = 0$ define um hiperplano no espaço n-dimensional. O neurônio atua como um separador linear: pontos de um lado ativam ($z > 0$), enquanto os do outro não.

Em 2D, isso é uma reta; em 3D, um plano. Para dados não linearmente separáveis, múltiplos neurônios combinados (em camadas) aproximam fronteiras curvas via teorema da aproximação universal (Cybenko, 1989). Isso explica por que um único neurônio resolve apenas classificações lineares simples, mas redes profundas lidam com complexidades como reconhecimento de imagens (Haykin, 2009).

Aqui está um diagrama ilustrando um neurônio artificial e sua fronteira de decisão:

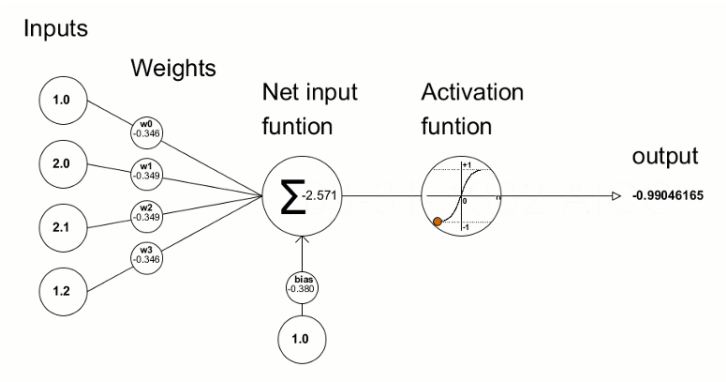

Figura 1: Diagrama de um neurônio artificial
Fonte: `https://wisertecnologia.com.br/blog/redes-neurais-artificiais/`

8 Implementação em Python

Para uma implementação básica sem bibliotecas:

```python
def neuronio(x, w, b):
    soma = 0
    for xi, wi in zip(x, w):
        soma += xi * wi
    z = soma + b
    return z  # Retorna z para aplicar ativacao separadamente

def degrau(z):
    return 1 if z > 0 else 0

def relu(z):
    return max(0, z)

def sigmoide(z):
    import math
    return 1 / (1 + math.exp(-z))

# Exemplo
x = [1, 0, 1]
w = [0.5, -0.3, 0.8]
b = 0.2
z = neuronio(x, w, b)
print(degrau(z))  # Saida: 1
print(relu(z))    # Saida: 1.5
print(sigmoide(z))  # Saida: approx0.817
```

Usando NumPy para eficiência em vetores grandes:

```python
import numpy as np

class Neuronio:
    def __init__(self, n_entradas):
        self.pesos = np.random.randn(n_entradas)
        self.bias = np.random.randn()

    def forward(self, x, ativacao='relu'):
        z = np.dot(self.pesos, x) + self.bias
        if ativacao == 'relu':
            return np.maximum(0, z)
        elif ativacao == 'sigmoide':
            return 1 / (1 + np.exp(-z))
        else:
            return z  # Sem ativacao
```

```
# Exemplo
neur = Neuronio(3)
saida = neur.forward(np.array([1, 0, 1]), 'relu')
print(saida)
```

Essa implementação escalável facilita experimentos com dados reais.

9 Camadas e Encadeamento

Neurônios são organizados em camadas, onde a saída de uma torna-se entrada da próxima:

$$\mathbf{a}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{a}^{(l)} + \mathbf{b}^{(l)})$$

Aqui, $\mathbf{W}^{(l)}$ é a matriz de pesos da camada l, e f é aplicada element-wise. Essa arquitetura feedforward permite representações hierárquicas: camadas iniciais extraem features básicas, enquanto profundas capturam abstrações (Goodfellow et al., 2016).

Em código NumPy:

```
def camada_forward(a_prev, W, b, ativacao='relu'):
    Z = np.dot(W, a_prev) + b
    if ativacao == 'relu':
        A = np.maximum(0, Z)
    # Adicionar outras ativacoes...
    return A
```

10 Grandes Modelos

Modelos como o GPT-3 possuem bilhões de parâmetros, organizados em dezenas de camadas com milhões de neurônios cada. O processo é:

1. Entrada tokenizada e convertida em vetores de embedding.
2. Propagação por camadas, com multiplicações matriciais e ativações (tipicamente ReLU ou GELU).
3. Atenção self-attention em transformers, que é uma extensão vetorial de neurônios.
4. Saída decodificada para texto humano.

Nem todos os neurônios ativam para toda entrada; padrões esparsos emergem, otimizando eficiência. Estudos mostram que escalar o número de neurônios melhora desempenho em tarefas como previsão de texto, conforme a lei de scaling de Kaplan et al. (2020).

11 Limitações e Considerações Práticas

Um único neurônio é limitado a separações lineares. Redes profundas mitigam isso, mas enfrentam desafios como overfitting ou exploding gradients. Inicialização de pesos (e.g., Xavier ou He) e regularização ajudam. Além disso, em hardware, operações vetoriais são aceleradas por GPUs, tornando viável o treinamento de grandes modelos (Bishop, 2006).

12 Conclusão

Apesar da sofisticação dos modelos atuais, sua fundação é simples: uma soma ponderada seguida de não-linearidade. Repetida em escala, essa estrutura permite a emergência de inteligência. Como resumido por Haykin (2009), multiplicação, soma e ativação são os pilares das redes neurais, incentivando experimentação para uma compreensão mais profunda.

Projetos e Código

Repositórios relacionados: - GitHub: `https://github.com/marcos-j-ferreira/neuron-from-scrat`
`git`

Referências

[1] GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning.* MIT Press, 2016.

[2] BISHOP, Christopher. *Pattern Recognition and Machine Learning.* Springer, 2006.

[3] HAYKIN, Simon. *Neural Networks and Learning Machines.* Pearson, 2009.

[4] McCULLOCH, Warren S.; PITTS, Walter. A Logical Calculus of the Ideas Immanent in Nervous Activity. *Bulletin of Mathematical Biophysics*, 1943.

[5] ROSENBLATT, Frank. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 1958.

[6] KAPLAN, Jared et al. Scaling Laws for Neural Language Models. arXiv preprint arXiv:2001.08361, 2020.